```
    result = base params...
    decoration params...
    result
```

The combinators `after` and `before` don't change the behavior of the function that they wrap. When you need to change the return value, or the invocation of the wrapped function, then you need to use `around`.

6.4.3. Around

This one is fun. Suppose that the database is changed so that you need to open and close the connection to the database for each operation you do. For every place in the program where you do a set of database operations, you need to put an `openConnection` and a `closeConnection` around them:

```
openConnection()
doSomethingToTheDb()
doSomethingElseToTheDb()
closeConnection()
```

You quickly tire of adding the same two lines of code throughout your program. Worse, sometimes the database fails to open, and you have to graft on a fix for that too:

```
dbConnectionIsOpen = openConnection()
if dbConnectionIsOpen
  doSomethingToTheDb()
  doSomethingElseToTheDb()
  closeConnection()
```

Instead of directly invoking the database open and close functions everywhere, you can use `around`:

```
around = (decoration) ->
  (base) ->
    (params...) ->
      callback = -> base params...
      decoration ([callback].concat params)...
```